Explanation of the caesar code:

Here is a clear and well-structured explanation:

Caesar Cipher Implementation

Introduction

The Caesar Cipher is one of the simplest and oldest encryption techniques. It is a type of substitution cipher in which each letter in the plaintext is shifted by a fixed number of positions in the alphabet.

For example, if the shift value is **3**, then:

- A becomes D
- B becomes E
- C becomes F

This program implements both **encryption** and **decryption** using the Caesar Cipher in Python.

1. Function: caesar_encrypt(text, shift)

This function encrypts the original message.

How it works:

1. **Initialize an empty string**

```
result = ""
```

This variable stores the encrypted text.

2. **Loop through each character**

```
for char in text:
```

The program checks every character in the input message.

3. **Check if character is uppercase**

```
if char.isupper():
    # Shift within uppercase ASCII range
    result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
```

It shifts the letter within the uppercase ASCII range (A–Z).

- Formula used:
- (ord(char) - ord('A') + shift) % 26 + ord('A')
- ord(char) converts the letter into its ASCII value.
- ord('A') is subtracted to bring it into a 0–25 range.
- shift is added.
- % 26 ensures wrap-around (Z goes back to A).
- chr() converts ASCII back to a character.

4. **Check if character is lowercase**

```
elif char.islower():
    # Shift within lowercase ASCII range
    result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
```

Same logic is applied, but within lowercase range (a–z).

5. **Other characters**

```
else:
    # Keep spaces and special characters unchange
    result += char
```

- Spaces, numbers, and special symbols are not changed.

6. **Return encrypted result**

```
return result
```

2. Function:

```
def caesar_decrypt(ciphertext, shift):  1 usage
```

This function reverses the encryption.

How it works:

- It follows the same logic as encryption.
- The only difference is that the shift value is **subtracted** instead of added:

(ord(char) - ord('A') - shift) % 26 + ord('A')

This restores the original message.

---

3. Main Program Execution

```
# Example usage
if __name__ == "__main__":
```

This ensures the program runs only when the file is executed directly.

Steps:

1. **User inputs message**
2. ```
   message = input("Enter message: ")
   ```
3. **User inputs shift value**
4. ```
   shift = int(input("Enter shift value: "))
   ```
5. **Encrypt the message**
6. ```
   encrypted = caesar_encrypt(message, shift)
   ```
7. **Print encrypted message**
8. ```
   print("Encrypted:", encrypted)
   ```
9. **Decrypt the message**
10. ```
    decrypted = caesar_decrypt(encrypted, shift)
    ```
11. **Print decrypted message**
    ```

```
print("Decrypted:", decrypted)
```

---

output

```
"C:\Users\Almas Jabeen\PyCharmMiscProject\.venv\Scripts
Enter message: Almas jabeen
Enter shift value: 4
Encrypted: Epqew nefiir
Decrypted: Almas jabeen

Process finished with exit code 0
```

Conclusion

This program demonstrates how the Caesar Cipher works using ASCII values and modular arithmetic. It maintains uppercase and lowercase letters separately and keeps special characters unchanged. The decryption function correctly reverses the encryption by subtracting the shift value.